DAY 27 OF 30

Faster Spark jobs

Where the time really goes.
And the fixes that matter most.

Shuffle **Joins** **File layout**

Keshav Maheshwari
Data Engineer | Databricks · AWS & Azure · Streaming · Applied AI

Swipe →

THE PROBLEM

A bigger cluster is the expensive way to go faster.

1	**The job is slow**	Double the nodes
2	**It is still slow**	Double them again
3	**The bill**	Four times higher
4	**The real cause**	One bad join

Most slow Spark jobs do not need more machines. **They need less wasted work.**

Keshav Maheshwari
Data Engineer | Databricks · AWS & Azure · Streaming · Applied AI

Swipe →

THE SIMPLE VERSION

Fewer, better-packed trucks.

A delivery firm does not get faster by buying more trucks if they leave half empty and drive across town for one parcel.

Fast Spark is the same: **move less data, fewer times, in sensible loads**.

✓ **Read less**	Skip data you do not need
✓ **Move less**	Avoid shuffling between machines
✓ **Pack well**	Right-sized files and partitions

Keshav Maheshwari
Data Engineer | Databricks · AWS & Azure · Streaming · Applied AI

Swipe →

HOW IT WORKS

The fixes, in order of payoff

1. **Filter and select early**, so less data flows through
2. **Broadcast small tables** in joins to avoid a shuffle
3. **Fix skew**: one giant key can stall a whole job
4. **Cache** only what you reuse more than once
5. **Compact files** and choose a sensible layout

Read the plan first

The Spark UI shows which stage takes the time. Tune that stage, not the whole job.

Keshav Maheshwari
Data Engineer | Databricks · AWS & Azure · Streaming · Applied AI

Swipe →

FOR ENGINEERS

What Fabric adds, and what to check

Native execution engine

A vectorised engine you enable in the environment. It speeds up many queries with no code change.

Resource profiles

writeHeavy is the default. Read-heavy profiles change V-Order and write settings for serving tables.

Runtime version

Runtime 2.0 is generally available and recommended for production. New workspaces still default to 1.3.

Autotune is not the answer on new runtimes

It only works on Runtime 1.2, so do not rely on it for current workloads.

Keshav Maheshwari
Data Engineer | Databricks · AWS & Azure · Streaming · Applied AI

Swipe →

THE HONEST PART

Tune the code or add compute?

TUNE FIRST	ADD COMPUTE
One stage dominates the run time	The plan is clean and data simply grew
A join or shuffle is the bottleneck	The job is already well partitioned
Thousands of small files are being read	The deadline is fixed and close

> **The catch:** caching everything feels safe and makes jobs slower. Cached data takes memory away from the work. Cache what you reuse, and release it after.

Keshav Maheshwari
Data Engineer | Databricks · AWS & Azure · Streaming · Applied AI

Swipe →

FROM REAL WORK

Tuning on real platforms

The same techniques, on three different Databricks workloads.

1. **Broadcast joins, partitioning and caching** cut average run time by 25%
2. **Liquid Clustering** improved layout on a petabyte-scale pipeline
3. **Daily volumes** there ran from hundreds of GB to several TB
4. **A migration** took one process from 2 hr 50 min to about 5 min

The pattern each time: **remove wasted work first.**

Keshav Maheshwari
Data Engineer | Databricks · AWS & Azure · Streaming · Applied AI

Swipe →

SAVE THIS

Faster Spark in five lines

1. **Read the plan** before changing anything
2. **Filter early**, select only what you need
3. **Broadcast** small tables in joins
4. **Fix skew** and small files
5. **Enable** the native engine and check the runtime

Tomorrow: AI on Fabric.
Asking your data questions in plain language.

Keshav Maheshwari
Data Engineer | Databricks · AWS & Azure · Streaming · Applied AI

Follow for Day 28